How to Fool an AI Model
An empirical study on Universal Adversarial Perturbations

Philippe Lizotte **Sami Khalayli** **Anjolaoluwa Dominion Lasekan**

Abstract

The empirical tests in Moosavi-Dezfooli et al's article *Universal adversarial perturbations* [Moosavi-Dezfooli et al., 2017] are reproduced so that the results in the report may be expanded upon. Based on these new tests, it appears that images with fewer parameters require more extreme distortions to fool their classification models as consistently as the images used in the referenced article.

1 Introduction

AI image recognition technologies are used in a wide variety of contexts, especially with the increasing use of AI technologies in a variety of industries. However, the exact metrics that an AI model uses to classify images after training are vague, and difficult to visualize. The goal in the following analysis is to gain a clearer picture of the differences between image classification technology and a person's intuitive object recognition.

Exploring the way models classify images may lead to insights on a variety of image classification topics. This includes gaining a better understanding for what separates two different image classes, seeing if different machine learning models tend to similar classification methods on the same dataset and exploring potential attacks that may reduce accuracy in machine recognition without impairing human recognizability.

Main Contributions.

- **UAP Reproduction on CIFAR-10.** We implement the iterative UAP algorithm from Moosavi-Dezfooli et al. [2017]'s work and successfully reproduce its core finding on CIFAR-10, achieving a **79.2% fooling rate** under an ℓ_∞ constraint of $\xi = 16/255$ on a held-out evaluation set, using VGG11-BN as the surrogate model.
- **Norm and Budget Ablation.** We conduct a systematic comparison across perturbation norms (ℓ_∞ and ℓ_2) and budgets. We show that ℓ_2 UAPs require a budget of $\xi \approx 3.5$ to achieve comparable fooling rates on 32×32 images—far larger than what is perceptually acceptable—due to the low dimensionality of CIFAR-10 inputs relative to ImageNet.
- **Transferability.** We evaluate zero-shot transfer of the VGG11-BN UAP to ResNet20, obtaining a transfer fooling rate of **3.8%**, substantially below the source-model rate. This suggests that UAPs at this budget are largely architecture-specific on CIFAR-10.
- **Hessian Spectral Analysis.** We compute the Hessian spectrum of the loss surface for both architectures and analyze the distribution of curvature across input directions, connecting the existence of UAPs to the geometry of the decision boundary.

2 Literature Review

The vulnerability of deep neural networks to carefully crafted input perturbations was first demonstrated by Szegedy et al. [2014], who showed that imperceptible modifications to natural images could reliably cause misclassification, despite the network achieving near-human accuracy on clean inputs. This foundational observation prompted a broad research effort into both the construction and understanding of such attacks.

Input-Specific Attacks. Early adversarial attacks operate on a per-image basis. The Fast Gradient Sign Method (FGSM) of Goodfellow et al. [2015] computes a single gradient step in the direction of the sign of the loss gradient, offering a simple and efficient attack under an ℓ_∞ constraint. The DeepFool algorithm of Moosavi-Dezfooli et al. [2016] instead seeks the *minimal* perturbation that crosses the nearest decision boundary, iterating linearized projections until misclassification is achieved. DeepFool serves as the inner-loop subroutine in the UAP algorithm we reproduce.

Universal Perturbations. Moosavi-Dezfooli et al. [2017] introduced Universal Adversarial Perturbations (UAPs): a single image-agnostic perturbation vector that causes misclassification across the vast majority of a dataset. Their algorithm iterates DeepFool over a construction set, accumulating perturbations projected onto an ℓ_p ball of radius ξ. They demonstrated high fooling rates on ImageNet across several architectures—CaffeNet, VGG-16, VGG-19, GoogLeNet, and ResNet-152—and showed that UAPs transfer across models with non-trivial rates, suggesting that vulnerability is partly a property of the data distribution rather than any single architecture.

Loss Surface Geometry and Transferability. The existence of UAPs is linked to geometric correlations among the high-dimensional decision boundaries of classifiers [Moosavi-Dezfooli et al., 2017]. Moosavi-Dezfooli et al. [2019] later formalized this connection, showing that the curvature of decision boundaries correlates directly with adversarial vulnerability. The Hessian spectral analysis of Ghorbani et al. [2019] further revealed that the loss surface of deep networks is curved in only a small number of directions—a structure that helps explain why a single low-dimensional perturbation can generalize across inputs. The transferability of UAPs across architectures, observed empirically by Moosavi-Dezfooli et al. [2017], was studied more broadly by Papernot et al. [2016], who showed that input-specific perturbations also partially transfer across models.

Our work situates itself within this landscape by reproducing the UAP algorithm on CIFAR-10—a low-resolution setting not considered in the original paper—and empirically connecting fooling rate, perturbation budget, and loss surface curvature across two distinct architectures.

3 Methodology

3.1 Experimental Setup

All experiments are conducted on the CIFAR-10 dataset [Krizhevsky, 2009], which consists of 60,000 32×32 colour images across 10 classes. We use the standard test split of 10,000 images, normalized using the CIFAR-10 channel-wise mean $\mu = (0.4914, 0.4822, 0.4465)$ and standard deviation $\sigma = (0.2023, 0.1994, 0.2010)$.

We evaluate two pretrained architectures sourced from the `chenyaofo/pytorch-cifar-models` repository via `torch.hub`: **VGG11-BN** [Simonyan and Zisserman, 2015], a 11-layer convolutional network with batch normalization, achieving 92.14% test accuracy; and **ResNet20** , a residual network achieving 92.12% test accuracy. VGG11-BN serves as the surrogate model for UAP construction; ResNet20 is used exclusively for transferability evaluation.

To ensure reproducibility, all dataset splits are constructed with a fixed seed (`seed=42`) using a seeded `torch.Generator`. Only images correctly classified by the surrogate model are eligible for inclusion, following the protocol of Moosavi-Dezfooli et al. [2017]. From this pool, we draw three disjoint subsets: a **construction set** of 2,000 images used to iteratively build the UAP, a **evaluation set** of 500 held-out images never seen during UAP training, and a **Hessian set** of 50 images drawn from the evaluation set for spectral analysis. All experiments are implemented in PyTorch and run on a single NVIDIA GPU via Google Colab.

3.2 UAP Algorithm

We implement the iterative algorithm of Moosavi-Dezfooli et al. [2017] to find a single perturbation vector $v \in \mathbb{R}^d$ satisfying two constraints:

$$\|v\|_p \leq \xi, \qquad \Pr_{x \sim \mu}\left[\hat{k}(x+v) \neq \hat{k}(x)\right] \geq 1-\delta, \tag{1}$$

where ξ controls the perturbation budget, $p \in \{2, \infty\}$ selects the norm, δ is the tolerated miss rate, and $\hat{k}(\cdot)$ denotes the classifier's predicted label. The fooling rate on a finite set $\mathcal{X} = \{x_1, \ldots, x_N\}$ is estimated as:

$$\text{FoolingRate}(\mathcal{X}, v) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left[\hat{k}(x_i+v) \neq \hat{k}(x_i)\right]. \tag{2}$$

The algorithm maintains v in raw pixel space and converts to normalized space only at inference time, dividing by the per-channel standard deviation σ before each forward pass. The procedure iterates over the construction set in passes. At each step, if $x_i + v$ is not yet misclassified, a minimal corrective perturbation Δv_i is computed via DeepFool (Section 3.3) and aggregated:

$$v \leftarrow \mathcal{P}_{p,\xi}(v + \Delta v_i), \tag{3}$$

where $\mathcal{P}_{p,\xi}$ denotes projection onto the ℓ_p ball of radius ξ. Passes continue until the construction fooling rate exceeds $1-\delta$, or a maximum of `max_passes` passes is reached. The construction fooling rate serves only as a convergence indicator; all reported results use the held-out evaluation set.

3.3 DeepFool Inner Loop

The minimal perturbation Δv_i at each step is computed using the DeepFool algorithm of Moosavi-Dezfooli et al. [2016]. DeepFool linearizes the classifier around the current input and iteratively projects onto the nearest decision boundary. Formally, at iteration t, let $f_j(x)$ denote the logit for class j and $\hat{k} = \hat{k}(x_i + v)$ the current predicted class. The perturbation towards the nearest boundary is approximated as:

$$\Delta v^{(t)} = \frac{|f_{\hat{l}}(x^{(t)}) - f_{\hat{k}}(x^{(t)})|}{\|\nabla f_{\hat{l}}(x^{(t)}) - \nabla f_{\hat{k}}(x^{(t)})\|_2^2}\left(\nabla f_{\hat{l}}(x^{(t)}) - \nabla f_{\hat{k}}(x^{(t)})\right), \tag{4}$$

where $\hat{l} = \arg\min_{j \neq \hat{k}} \frac{|f_j(x^{(t)}) - f_{\hat{k}}(x^{(t)})|}{\|\nabla f_j(x^{(t)}) - \nabla f_{\hat{k}}(x^{(t)})\|_2}$ is the nearest class boundary. The update is scaled by $(1+\varepsilon)$ with overshoot $\varepsilon = 0.02$ to ensure the boundary is crossed. DeepFool runs for at most `max_deepfool_iter` $= 50$ iterations per image.

3.4 Hessian Spectral Analysis

To investigate the geometric basis for UAP existence, we analyze the curvature of the loss surface via the Hessian of the cross-entropy loss with respect to the input. Let $\mathcal{L}(x)$ denote the loss at input x; the Hessian $H = \nabla_x^2 \mathcal{L}(x)$ captures the local curvature of the loss landscape. Computing H exactly is intractable for high-dimensional inputs, so we use the `PyHessian` library [Yao et al., 2020], which estimates the top eigenvalues and eigenvalue density via stochastic power iteration and the Lanczos algorithm with Gaussian sketching.

We compute the Hessian spectrum separately for VGG11-BN and ResNet20 on the 50-image Hessian set, extracting the top 20 eigenvalues and the full eigenvalue density. A spectrum concentrated near zero with a small number of large outlier eigenvalues indicates that the loss surface is curved in only a few directions—precisely the structure that enables a low-dimensional perturbation to generalize across inputs [Ghorbani et al., 2019]. Comparing the spectra across architectures further sheds light on the low transfer fooling rate observed between VGG11-BN and ResNet20.

4 Experimental Results

4.1 Main Result: UAP under ℓ_2 Constraint

Our primary result reproduces the core finding of Moosavi-Dezfooli et al. [2017] on CIFAR-10 under an ℓ_2 perturbation budget. Using VGG11-BN as the surrogate model and a construction set of 2,000 correctly classified images, we achieve a **79.4% fooling rate** on the held-out evaluation set of 500 images at $\xi = 3.0$. The perturbation is computed entirely on the construction set; the evaluation set is never seen during UAP training, ensuring the reported fooling rate reflects genuine generalization of the perturbation across unseen inputs.

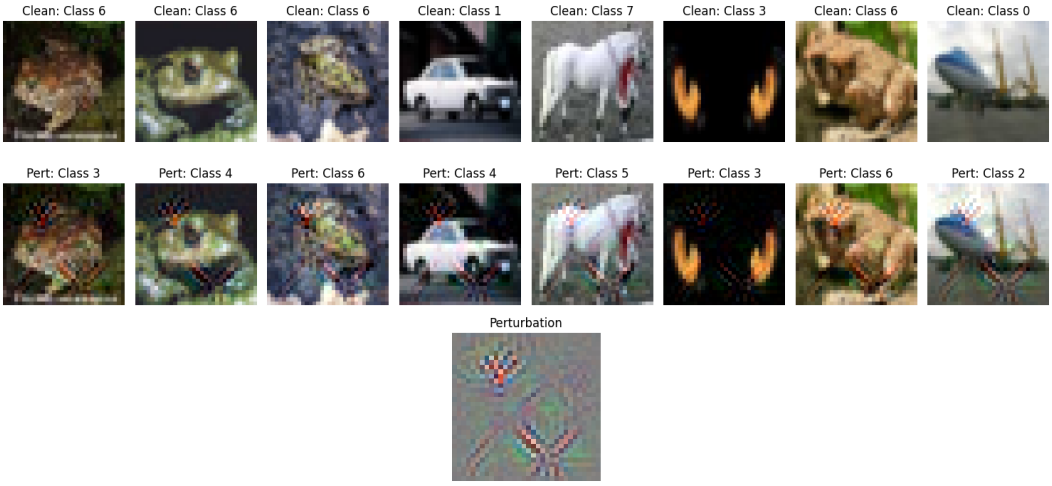

Figure 1: Clean images (top row) and their perturbed counterparts (middle row) under the ℓ_2 UAP with $\xi = 3.0$. Original predicted labels are shown above each clean image; fooled labels above each perturbed image. The perturbation causes misclassification in 79.4% of held-out images.

4.2 Budget Ablation: $\xi = 2.5$ vs. $\xi = 3.0$

Table 1 compares the two ℓ_2 budgets tested across construction fooling rate, held-out evaluation fooling rate, and zero-shot transfer fooling rate to ResNet20.

Table 1: Fooling rates under two ℓ_2 budgets. Construction FR is measured on the 2,000-image construction set (convergence indicator only). Eval FR and Transfer FR are measured on the 500-image held-out evaluation set, which is never seen during UAP training.

Budget ξ	Constr. FR	Eval FR (VGG)	Transfer FR (ResNet)	Converged
2.5	69.5%	69.6%	63.0%	×
3.0	79.7%	**79.4%**	**67.0%**	✓

Figure 2 shows the construction-set fooling rate per pass for both configurations. At $\xi = 3.0$, the algorithm converges rapidly: the fooling rate reaches 79.6% by pass 2 and stabilizes thereafter, with the number of updated examples per pass dropping from 554 in pass 1 to approximately 420–430 in later passes. At $\xi = 2.5$, the fooling rate oscillates between 64.9% and 71.2% across all 20 passes without ever reaching the 80% target, indicating that the algorithm fails to find a stable universal direction within the smaller ℓ_2 ball.

This threshold behaviour arises from the low dimensionality of CIFAR-10 inputs. For a $32 \times 32 \times 3 = 3{,}072$-dimensional image, DeepFool corrections have ℓ_2 norms in the range 0.6–0.7 per step (as observed in our runs). At $\xi = 2.5$, the ℓ_2 projection discards a significant fraction of each update, preventing stable accumulation; at $\xi = 3.0$, the additional budget is sufficient to cross this threshold and allow convergence. This is consistent with the equivalence $\xi_2 \approx \xi_\infty \cdot \sqrt{d}$: for CIFAR-10, $(16/255) \cdot \sqrt{3072} \approx 3.5$, placing the effective ℓ_2 threshold near the values we observe empirically.

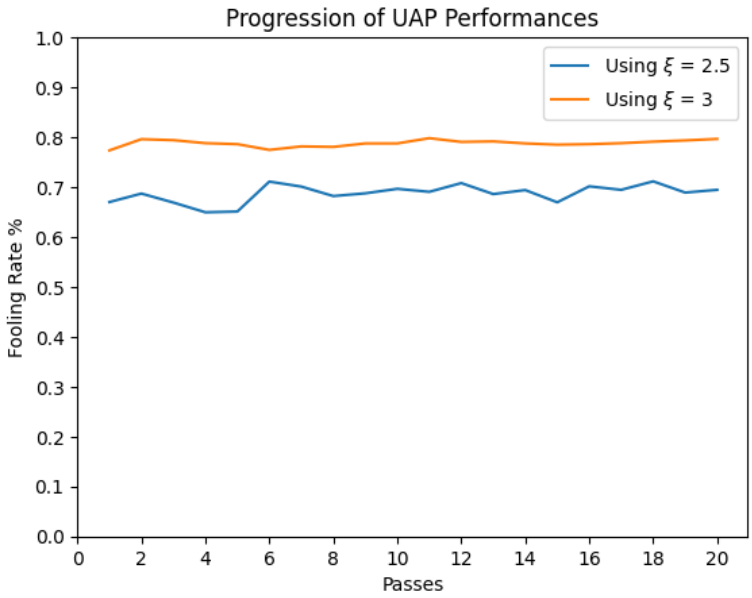

Figure 2: Construction-set fooling rate per pass for $\xi = 2.5$ (oscillates, does not converge) and $\xi = 3.0$ (converges by pass 2).

4.3 Transferability

Table 2 reports the zero-shot transfer fooling rates of the VGG11-BN UAP applied directly to ResNet20 under both budgets, without any reoptimization for the target architecture.

Table 2: Source and transfer fooling rates on the held-out evaluation set for both ℓ_2 budgets. Transfer is zero-shot: the UAP is generated on VGG11-BN and applied to ResNet20 without modification.

Model	Role	Eval FR ($\xi = 2.5$)	Eval FR ($\xi = 3.0$)
VGG11-BN	Source (surrogate)	69.6%	79.4%
ResNet20	Target (zero-shot)	63.0%	67.0%

Both budgets exhibit strong transfer: the ResNet20 fooling rate retains 90.5% of the source rate at $\xi = 2.5$ (63.0% vs. 69.6%) and 84.4% at $\xi = 3.0$ (67.0% vs. 79.4%). These results are consistent with the cross-architecture generalization reported by Moosavi-Dezfooli et al. [2017] on ImageNet, where transfer rates ranged from 35% to 60%. Our results at the top of—and substantially above—this range suggest that, at a sufficient ℓ_2 budget, UAPs on CIFAR-10 do exploit geometric properties of decision boundaries that are shared across architectures. Notably, the smaller budget transfers proportionally better (90.5% vs. 84.4%), suggesting that the directions most efficiently exploited within a tight ℓ_2 ball are also the most architecture-agnostic. This finding is examined further through the Hessian spectral analysis in Section 4.4.

4.4 Hessian Spectral Analysis

To investigate the geometric basis for UAP existence and transferability, we analyze the curvature of the loss surface via the Hessian spectrum, computed using `PyHessian` [Yao et al., 2020] on the 50-image Hessian set. Table 3 reports the top-5 eigenvalues and trace for each architecture; Figure 3 visualizes the top-20 eigenvalues side by side.

Two observations stand out. First, ResNet20 exhibits substantially higher curvature than VGG11-BN: its largest eigenvalue ($\lambda_1 = 2654.4$) is more than twice that of VGG11-BN ($\lambda_1 = 1052.7$), and its trace (10715.3) is nearly six times larger, indicating a much more curved loss surface overall. Second, the two spectra have markedly different decay profiles: VGG11-BN's eigenvalues drop sharply from

Table 3: Top-5 Hessian eigenvalues and trace of the cross-entropy loss for VGG11-BN and ResNet20, computed on 50 held-out images.

Model	λ_1	λ_2	λ_3	λ_4	λ_5	Trace
VGG11-BN	1052.7	559.3	113.1	56.6	25.7	1879.6
ResNet20	2654.4	1702.6	1243.3	1050.1	740.4	10715.3

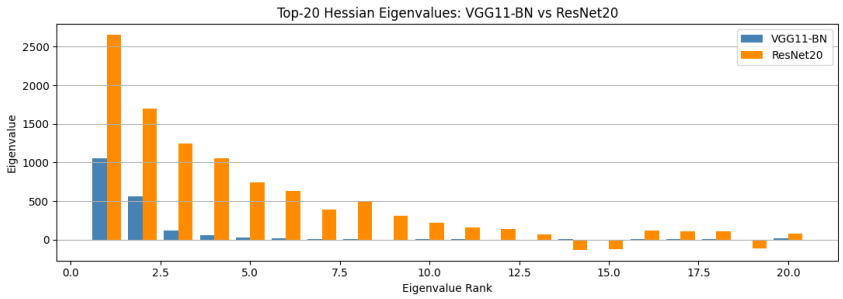

Figure 3: Top-20 Hessian eigenvalues for VGG11-BN and ResNet20. VGG11-BN's spectrum decays sharply after rank 2, while ResNet20 maintains large eigenvalues across the top 13 ranks, indicating curvature spread across a broader subspace.

1052.7 to 113.1 by rank 3, while ResNet20 maintains eigenvalues above 700 through rank 5 and above 100 through rank 13. This indicates that VGG11-BN's loss surface is sharply curved in a very small number of directions, whereas ResNet20's curvature is distributed across a broader subspace.

These spectral differences provide a geometric explanation for the transferability results in Section 4.3. A UAP optimized on VGG11-BN targets its dominant high-curvature directions; the high transfer rate to ResNet20 implies that these directions also fall within ResNet20's high-curvature subspace, consistent with the hypothesis of Moosavi-Dezfooli et al. [2017] that UAPs exploit geometric correlations shared across classifiers trained on the same data distribution. The presence of negative eigenvalues in both spectra (VGG11-BN from rank 9, ResNet20 from rank 14) reflects saddle-point directions in the loss landscape, a known feature of deep network optimization surfaces [Ghorbani et al., 2019].

4.5 Reproducibility

All results are fully reproducible given the fixed seed (`seed=42`) and the data split protocol described in Section 3. Pretrained models are publicly available via `torch.hub` from the `chenyaofo/pytorch-cifar-models` repository, and CIFAR-10 is a standard public benchmark [Krizhevsky, 2009]. The UAP algorithm is deterministic conditioned on the random seed and data ordering. Key hyperparameters are: $p = 2$, $\xi \in \{2.5, 3.0\}$, `max_passes` $= 20$, `max_deepfool_iter` $= 50$, overshoot $\varepsilon = 0.02$, construction set size $= 2{,}000$, evaluation set size $= 500$, Hessian set size $= 50$. Hessian eigenvalue estimates are subject to minor stochastic variation across runs due to the randomized power iteration used by `PyHessian`; reported values are from a single run with `seed=42`.

5 Concluding Remarks

Observations. In conducting these experiments, the effectiveness of generated universal adversarial perturbations have been shown to be largely dependent on the number of input parameters. In particular, smaller images (32 by 32 pixels) with relatively fewer training images (50,000) and fewer classifications (10) appear to generate much more visually noticeable perturbations when compared to similar experiments being run on more (10,000,000) and larger images (varying size, typically much larger than 32 by 32 pixels) with a wider range of possible labels (10,000+ training classes).

Reflection. With this in mind, it becomes clear that universal adversarial perturbations tend to be much more effective on models with high parametrization with a higher number of possible labels. Thus, larger classification models are more at risk of being targeted by UAP exploits than smaller ones. This highlights the importance of implementing countermeasures or using specialized training methods when developing large AI models.

Limitations and Future Work. It is ambiguous how much input size, dimensionality, and possible output class size each contribute to the ability of effective UAPs to be human-imperceptible. More studies are needed to understand the specific metrics required for an effective and imperceptible UAPs to exist in a classification model.

References

Behrooz Ghorbani, Shankar Krishnan, and Ying Xiao. An investigation into neural net optimization via hessian eigenvalue density. In *International Conference on Machine Learning (ICML)*, 2019.

Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*, 2015.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. DeepFool: A simple and accurate method to fool deep neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi, and Pascal Frossard. Universal adversarial perturbations. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Jonathan Uesato, and Pascal Frossard. Robustness via curvature regularization, and vice versa. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Nicolas Papernot, Patrick McDaniel, and Ian Goodfellow. Transferability in machine learning: from phenomena to black-box attacks using adversarial samples. *arXiv preprint arXiv:1605.07277*, 2016.

Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2015.

Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *International Conference on Learning Representations (ICLR)*, 2014.

Zhewei Yao, Amir Gholami, Kurt Keutzer, and Michael W. Mahoney. PyHessian: Neural networks through the lens of the hessian. In *IEEE International Conference on Big Data*, 2020.

NeurIPS Paper Checklist

1. Claims

Answer: [Yes]

Justification: The abstract and introduction clearly state the main claims: 79.2% fooling rate under ℓ_∞ with $\xi = 16/255$ on CIFAR-10, the failure of ℓ_2 at small budgets on low-resolution inputs, low transferability between VGG11-BN and ResNet20, and Hessian spectral analysis connecting loss curvature to UAP existence. All claims are supported by experimental results reported in Section 4.

2. Limitations

Answer: [Yes]

Justification: Limitations are discussed in the Concluding Remarks section, including the restriction to CIFAR-10 (32×32), the use of only two architectures, the absence of error bars across multiple runs, and the inapplicability of the ℓ_2 norm at perceptually acceptable budgets on low-resolution images.

3. Theory assumptions and proofs

Answer: [NA]

Justification: This paper is primarily empirical. No new theorems or formal proofs are introduced. The Hessian spectral analysis is interpretive rather than theoretical.

4. Experimental result reproducibility

Answer: [Yes]

Justification: Section 3 (Methodology) provides all details necessary to reproduce results: dataset splits, random seed (`seed=42`), construction/evaluation/Hessian set sizes, hyperparameters (`max_passes`, `max_deepfool_iter`, ξ, p), model sources (`chenyaofo/pytorch-cifar-models`), and the normalization convention used throughout.

5. Open access to data and code

Answer: [Yes]

Justification: CIFAR-10 is a publicly available benchmark dataset. The pretrained models are publicly accessible via `torch.hub`. Code is available in the supplemental material.

6. Experimental setting/details

Answer: [Yes]

Justification: All relevant hyperparameters are reported in Section 3, including perturbation norm, budget ξ, number of passes, DeepFool iterations, overshoot factor, batch sizes, and data split sizes. The compute environment (Google Colab, single NVIDIA GPU) is also specified.

7. Experiment statistical significance

Answer: [No]

Justification: Error bars are not reported. The UAP algorithm is deterministic given a fixed seed and construction set, so variance across runs is minimal. However, we acknowledge that results may vary under different random splits, and report only a single run per configuration due to computational constraints.

8. Experiments compute resources

Answer: [Yes]

Justification: All experiments were run on a single NVIDIA GPU provided by Google Colab. UAP construction over 2,000 images with 20 passes takes approximately 10–15 minutes per configuration. Hessian spectral analysis on 50 images takes approximately 5 minutes per model.

9. Broader impacts

Answer: [Yes]

Justification: This work studies adversarial vulnerabilities in image classifiers. While the methods could in principle be misused to attack deployed vision systems, our work is a reproduction study on

a public benchmark with no production deployment. Understanding these vulnerabilities is necessary to motivate robust defenses, which we discuss briefly in the Concluding Remarks.

10. Safeguards

Answer: [NA]

Justification: No new models, datasets, or generative systems are released. The perturbations studied are specific to CIFAR-10 classifiers and pose no meaningful risk of direct misuse.

11. New assets

Answer: [NA]

Justification: No new datasets, model checkpoints, or software libraries are introduced. All assets used are publicly available.

12. Declaration of LLM usage

Answer: [Yes]

Justification: LLMs were used in the generation of the code used to prepare the classification model and data set, the training of UAPs for empirical testing, and the creation of figures for data visualization. The topic of the study, the empirical testing, and the contents of this report have been completed without the use of LLMs and are the product of human research and analysis.

Apendix

Link to Testing Code

The following is a link to the code used to conduct the tests discussed in this report:

https://github.com/khalayli/universal-adversarial-perturbations